

# Fix: Treatment links without detail pages

---

Type: Fix Status: verified Branch: `fix/treatment-links-without-detail-pages`

## The problem

---

`serviceHref` in `lib/services.ts` returns `/treatments/<segment>/<slug>` for every one of the 40 services, unconditionally. No such route exists, and after feature 5 only ten to twelve of them will.

The header builds its dropdowns from that helper, so the site currently ships **40 dead treatment links in the main nav on every page**. `components/home/Signature.tsx` hardcodes a 41st, `/treatments/skin/microneedling-face`, on the signature treatment call to action.

The plan already says thin treatments should point at their card on the category page instead. Nothing implements it: there is no flag marking which treatments earn a page, `serviceHref` has no branch, and `ServiceCard` renders no anchor to land on.

Left alone this does not resolve when feature 5 ships. Feature 5 adds ten to twelve pages, so roughly twenty-eight to thirty of those links stay broken permanently.

## The fix

---

One flag, one branch, one `id`.

- Add `detailPage?: true` to `Service`, optional in the same style as `eminence` and `variantOf`, so no existing entry has to change.
- `serviceHref` branches: flagged services keep `/treatments/<segment>/<slug>`; everything else returns `/treatments/<segment>#<slug>`.
- `ServiceCard` renders `id={service.slug}` with `scroll-mt-24`, matching the offset the contact page already uses for `#form`, so a header-height sticky bar does not cover the card it just jumped to.
- `Signature.tsx` calls `serviceHref(getService("microneedling-face"))` rather than a hardcoded string.

**Ship it with zero services flagged.** That is the point of the shape: today all 41 links become anchors and nothing 404s, and feature 5 flips a flag to `true` in the same commit that adds each page. The link follows the flag, so the site is correct at every stage of a rollout that happens gradually as copy gets written. It also means this fix needs no decision about which treatments deserve a page, which is the part that would otherwise block it.

Must not break: the five category URLs, the eight concern URLs, the category links on concern page headings, or the `/contact?treatment=` links on the cards. Those come from `categoryHref`, `concernHref` and the card action, none of which this touches.

## Build steps

---

- ☒ **1. Flag, branch and anchor.** Add `detailPage` to the `Service` type, branch `serviceHref`, and give `ServiceCard` its `id` and scroll offset. *Done when:* `npm run build` is clean and still lists the same 18 routes; every card on `/treatments/facials` carries an `id` equal to its slug; and every href the header generates for a service is an anchor of the form `/treatments/<segment>#<slug>` with no service returning a nested path while nothing is flagged.
- ☒ **2. The hardcoded home link.** Point `Signature.tsx` at `serviceHref`. *Done when:* the signature call to action resolves to `/treatments/skin#microneedling-face`, and grepping the app for the string `/treatments/` finds it only in `categoryHref` and `serviceHref`.
- ☒ **3. Added during the build: the two leaked labels.** Frank pointed out that the header already calls this "What to treat", agreed earlier precisely because "concern" reads as negative, and that the old word had crept back into visible copy. Two strings, both mine or inherited from the prototype. *Done when:* the home eyebrow reads "What to treat" instead of "Start with the concern", the concern page's closing block reads "Also treating" and "What else we treat" instead of "Something else bothering you" and "Browse another concern", and no visitor-facing string in the app contains the word. The code, the type names and the URL keep using `concern`; only copy changed.
- ☒ **4. Added during the build: rename the route to /treat.** Frank asked for it once the label mismatch surfaced. The folder said `concerns` while the header said "What to treat", and the URL is the one place the old word would have been expensive to change later. Free now because nothing is deployed and nothing is indexed, so no redirects are owed. *Done when:* `app/concerns/[slug]` is `app/treat/[slug]`, `concernHref` returns `/treat/<slug>`, all eight pages answer 200 at the new path, and the home chips follow without being touched. Code, types, data files and the component folder keep the `concern` name; only the public URL changed.

## Verify

---

Against the production server, since `dynamicParams` and static output are what this affects:

1. Build, restart `next start`, and collect every href beginning `/treatments/` from the rendered header on any page.
2. Fetch each distinct one. All must return 200, and there must be 41 distinct service anchors plus the 5 category pages, with zero 404s.
3. For a sample of anchors, confirm the fragment matches an element `id` in the returned HTML, so the link lands on a card rather than the top of the page.
4. Click one in the browser and confirm the card is visible below the sticky header, not hidden behind it.
5. Regression: the five category pages, eight concern pages, `/contact` and `/` all still return 200, and `/contact?treatment=laser-hair-removal` still prefills.

## Implementation walkthrough

---

**The flag** ( `types/services.ts` ) - `detailPage?: true` , optional in the same style as `eminence` and `variantOf` , so not one of the forty existing entries had to change. It means published, not planned: a treatment carries it once it has its own page with real content behind it.

**The branch** ( `lib/services.ts` ) - `serviceHref` returns the nested route for a flagged service and `/treatments/<segment>#<slug>` for everything else. The header already routed every treatment through this helper, so the whole nav changed behaviour without the nav being touched. Shipped with zero services flagged, which is the point: forty links start working today, and feature 5 flips a flag in the same commit that adds each page.

**The landing spot** ( `components/treatments/ServiceCard.tsx` ) - `id={slug}` plus `scroll-mt-24` , the same offset the contact page uses for `#form` , so the sticky header does not cover the card the nav just jumped to.

**The hardcoded one** ( `components/home/Signature.tsx` ) - the signature call to action pointed at a literal `/treatments/skin/microneedling-face` and had been a 404 since it was written. It now calls `serviceHref` on the service it already resolves as `lead` .

**The words** ( `components/home/Concerns.tsx` , the concern page) - Frank pointed out that the header has said "What to treat" since the header commit, agreed that way because "concern" reads as negative, and that the old word had crept back into two visible strings: an eyebrow inherited from the prototype and a heading written the day before in feature 6. Now "What to treat" and "What else we treat". No visitor-facing string in the app contains the word.

**The URL** - `app/concerns/[slug]` became `app/treat/[slug]` and `concernHref` returns `/treat/<slug>` . One line of source produced the URL, which is why the rename touched exactly two things. Done now because nothing is deployed and nothing is indexed, so it costs no redirects; after launch it would have. The code, types, data file and component folder still say `concern` , deliberately: no visitor sees them and renaming them would be churn.

**Verified** - `npx tsc --noEmit` , `npm run lint` , `npm run build` clean, 18 routes. All forty treatment links resolve to a real card `id` on the right category page, checked by generating every href from the data and matching it against the built HTML: 40 of 40, zero broken, zero flagged. No nested treatment path remains anywhere in source; only `categoryHref` and `serviceHref` build treatment URLs. In the browser, `/treatments/facials#deep-cleansing-facial` scrolls to that card and clears the header. The home signature link is now `/treatments/skin#microneedling-face` . All eight pages answer 200 at `/treat/<slug>` , the old `/concerns/` paths are gone, the home chips point at the new URLs without being edited, and `/` , `/contact` and the category pages are unaffected.

One caveat worth recording: an earlier check of mine reported a leaked word on the home page. That was a false positive from a loose regex matching the RSC payload inside a `<script>` tag, not visible copy. Exact string checks confirmed all three labels are correct.